

08.3 — Regression Test Script (8-Phase Playbook)

For: Admin / Venkat running a full regression test on Haritha Hospitals ERPNext.
Goal: Single source of truth for the 8-phase test playbook (LEARNINGS #48) — copy, run, report. **Last updated:** 2026-08-29 **Pre-req reading:** 08.1 Test Plan, 08.2 Test Cases, LEARNINGS #48

Purpose

The 8-phase regression script is the **mandatory test flow** for any major change:

- New module / app install
- Schema migration
- Quarterly regression
- Pre-upgrade validation
- Post-incident verification

Total runtime: ~4 hours for full regression; ~1.5 hours for 8-phase lite.

Venkat's standing rule (LEARNINGS #48, set 2026-08-10): After ANY new module/app/config is installed, ALWAYS proactively offer to run this 8-phase playbook. Non-negotiable.

When to run

Trigger	Plan	Time
bench install-app <X>	Full 8-phase	~4 hours
Schema change (CF/PS/PF/LH on critical DocType)	Lite (Phases 1-6)	~1.5 hours
Quarterly cadence	Full 8-phase + Phase 7 (60+ cases)	1 day
Pre-upgrade (Frappe/ERPNext/HRMS major version)	Full 8-phase	~4 hours
Post-incident verification	Affected phases only	~30 min
Deploy to prod	Smoke (Phase 1) + targeted phases	~15 min

Phase 1 — Smoke (~5 min)

Goal: System boots, UI loads, no 500 errors.

Curl tests (HTTP 200)

```
# All 3 envs respond?  
for env in dev qa prod; do  
  case $env in  
    dev) url="https://pberpDEV.duckdns.org" ;;
```

```

    qa)    url="https://pberpqa.duckdns.org" ;;
    prod)  url="https://pberpprod.duckdns.org" ;;
esac
echo -n "$env: "
curl -sk -o /dev/null -w "%{http_code}\n" "$url"
done
# Expected: 200 200 200

```

Login test

```

# Login as Administrator via API
curl -sk -c /tmp/cookies.txt \
  -X POST "https://pberpprod.duckdns.org/api/method/login" \
  -d "usr=administrator&pwd=$ADMIN_PASS"
# Expected: HTTP 200, JSON with home_page URL

# Verify session
curl -sk -b /tmp/cookies.txt \
  "https://pberpprod.duckdns.org/api/method/frappe.auth.get_logged_user"
# Expected: "administrator"

```

List views load

```

# Hit key DocType list endpoints
for doctype in Employee Shift Type Attendance Leave Application Salary Slip; do
  echo -n "$doctype: "
  curl -sk -b /tmp/cookies.txt \
    "https://pberpprod.duckdns.org/api/resource/$doctype?fields=%5B%22name%22%5D&limit_page
    | jq -r '.data | length' \
    | head -1
done
# Expected: 1 (at least 1 record exists)

```

Pass criteria for Phase 1

- ☐ All curl tests return 200
- ☐ Login succeeds
- ☐ All 5 DocType list endpoints respond
- ☐ No errors in browser console

If any smoke fails: Stop. Investigate before proceeding.

Phase 2 — Vendor tests (~30 min)

Goal: Run the codebase's own test suite — free win.

```

# Haritha custom app tests
bench --site pberpprod.duckdns.org run-tests --app haritha_hospital 2>&1 | tee /tmp/vendor

# HRMS core tests (smoke subset)
bench --site pberpprod.duckdns.org run-tests --app hrms --module hrms.tests.test_attendance

```

Important caveats (LEARNINGS #55, #56, #58):

- If allow_tests=false in site_config.json, prefix with CI=1:
`CI=1 bench --site pberpprod.duckdns.org run-tests --app haritha_hospital`
- Bootstrap blockers (FY overlap, stale records) — see 04.4 Incident Response for fixes
- Scope by module/doctype if full suite has too many failures: `--module <X>` or `--doctype <Y>`

Pass criteria for Phase 2

- ☐ No Python tracebacks
 - ☐ Pass rate > 90% (some pre-existing failures OK, document them)
 - ☐ No new failures vs. last green run
-

Phase 3 — Schema verification (~15 min)

Goal: All expected customizations present and correctly tagged.

Count customizations

`bench --site pberpprod.duckdns.org console`

In console:

`import frappe`

```
cf = frappe.db.count("Custom Field", filters={"module": "haritha_hospital"})
ps = frappe.db.count("Property Setter", filters={"module": "haritha_hospital"})
pf = frappe.db.count("Print Format", filters={"module": "haritha_hospital"})
n = frappe.db.count("Notification", filters={"module": "haritha_hospital"})
lh = frappe.db.count("Letter Head", filters={"module": "haritha_hospital"})

print(f"CF: {cf} (expect 78)")
print(f"PS: {ps} (expect 189)")
print(f"PF: {pf} (expect 3)")
print(f"N: {n} (expect 2)")
print(f"LH: {lh} (expect 2)")
```

Verify haritha_hospital app installed

```
frappe.db.get_value("Installed Applications", {"app_name": "haritha_hospital"}, "app_version")
# Expected: a version string like "0.0.1"
```

Sanity: no orphan customizations

```
# CF that reference missing DocType
orphan_cf = frappe.db.sql("""
    SELECT name, dt FROM `tabCustom Field`
    WHERE module = 'haritha_hospital'
    AND dt NOT IN (SELECT name FROM `tabDocType`)
""")
print(f"Orphan CFs: {len(orphan_cf)}") # Expected: 0
```

```
# PS that reference missing DocType
orphan_ps = frappe.db.sql("""
    SELECT name, doc_type FROM `tabProperty Setter`
    WHERE module = 'haritha_hospital'
    AND doc_type NOT IN (SELECT name FROM `tabDocType`)
""")
print(f"Orphan PS: {len(orphan_ps)}") # Expected: 0
```

Pass criteria for Phase 3

- ☐ All counts match expected (78/189/3/2/2)
- ☐ haritha_hospital in Installed Applications
- ☐ 0 orphan customizations

If counts differ: investigate missing customizations. Re-export fixtures if needed.

Phase 4 — CRUD (~30 min)

Goal: Create / read / update / delete for each major DocType works.

Test fixtures (idempotent)

bench --site pberpprod.duckdns.org execute haritha_hospital.tests.fixtures.seed_smoke_data

Fixture file: haritha_hospital/tests/fixtures/smoke_data.py — creates 1 Employee, 1 Shift Type, 1 Holiday, 1 Leave Allocation, 1 Salary Structure.

Manual CRUD checks

For each DocType below, run a quick CRUD via API:

Employee

```
# Create
curl -sk -b /tmp/cookies.txt -X POST \
  "https://pberpprod.duckdns.org/api/resource/Employee" \
  -H "Content-Type: application/json" \
  -d '{
    "doctype": "Employee",
    "first_name": "Test",
    "last_name": "Smoke",
    "company": "Haritha Hospitals",
    "department": "Cardiology",
    "designation": "Staff Nurse",
    "date_of_joining": "2026-08-29"
  }'

# Read
curl -sk -b /tmp/cookies.txt \
  "https://pberpprod.duckdns.org/api/resource/Employee/Test Smoke"

# Update
curl -sk -b /tmp/cookies.txt -X PUT \
```

```
"https://pberpprod.duckdns.org/api/resource/Employee/Test Smoke" \
-H "Content-Type: application/json" \
-d '{"cell_number": "+91-9999999999"}'
```

Delete

```
curl -sk -b /tmp/cookies.txt -X DELETE \
  "https://pberpprod.duckdns.org/api/resource/Employee/Test Smoke"
```

Repeat the same pattern for:

- Shift Type (T-SHIFT-001)
- Shift Assignment (T-SHIFT-004)
- Attendance (T-ATT-001)
- Leave Application (T-LEAVE-001)
- Salary Slip (T-PAY-001)

Pass criteria for Phase 4

- ☐ All 6 DocTypes: Create returns 200 with new doc name
 - ☐ All 6: Read returns expected fields
 - ☐ All 6: Update succeeds
 - ☐ All 6: Delete succeeds (or returns expected link-integrity error)
-

Phase 5 — Workflow (~30 min)

Goal: Submit / approve / cancel flows work end-to-end.

Submit attendance → verify approved

Submit

```
curl -sk -b /tmp/cookies.txt -X POST \
  "https://pberpprod.duckdns.org/api/method/frappe.client.submit" \
  -d "doc=$(echo -n '{"doctype":"'Attendance',"name":"'HR-ATT-...'}' | python3 -c 'i
```

Verify status

```
frappe.db.get_value("Attendance", "HR-ATT-...", "docstatus")
# Expected: 1 (Submitted)
```

Apply leave → approve → verify balance

1. *Apply (use UI or API)*

2. *Login as manager, approve via ToDo or directly*

3. *Verify:*

```
bench --site pberpprod.duckdns.org console
```

```
>>> frappe.db.get_value("Leave Application", "<id>", "status")
```

Expected: "Approved"

```
>>> frappe.db.get_value("Leave Allocation", {"employee": "<X>", "leave_type": "Casual Leav
```

Mark shift assignment → verify on roster

1. *Create Shift Assignment via API*

2. *Open /hr/roster*

3. *Visual verify (manual)*

Cancel + amend test

```
# Cancel a submitted doc
curl -sk -b /tmp/cookies.txt -X POST \
  "https://pberpprod.duckdns.org/api/method/frappe.client.cancel" \
  -d "doc=..."

# Amend
# (UI only – DocType-specific flow)
```

Pass criteria for Phase 5

- ☐ Submit → status = 1
 - ☐ Approve → status updates + ledger entry created
 - ☐ Cancel → status = 2, audit entry
 - ☐ Amend → new doc with same date
-

Phase 6 — Integration (~30 min)

Goal: Cross-module flows work.

Checkin → attendance → payroll (end-to-end)

```
bench --site pberpprod.duckdns.org console
```

```
import frappe
from frappe.utils import today, add_days

emp = "Test Smoke" # use the fixture employee
date = today()
```

```
# 1. Insert Employee Checkin
```

```
checkin = frappe.new_doc("Employee Checkin")
checkin.employee = emp
checkin.time = f"{date} 09:00:00"
checkin.log_type = "IN"
checkin.insert()
checkin.submit()
```

```
# 2. Force auto-attendance run
```

```
from hrms.hr.doctype.attendance.attendance import mark_attendance
```

```
# Or trigger the scheduled job:
```

```
frappe.flags.in_test = True
```

```
from frappe.utils.scheduler import enqueue_events
```

```
# Simpler: directly create Attendance
```

```
att = frappe.new_doc("Attendance")
att.employee = emp
att.attendance_date = date
att.status = "Present"
att.insert()
att.submit()
print(f"Attendance created: {att.name}")
```

3. Verify payroll sees it

```
ss_count = frappe.db.count("Salary Slip", {"employee": emp, "start_date": ["<=", date], "e
print(f"Related Salary Slips: {ss_count}")
```

Roster SPA loads + filters work

Manual:

1. Open /hr/roster
2. Filter by Department = Cardiology
3. Verify only Cardiology employees shown
4. Click a shift cell → employee profile opens
5. Browser console: 0 errors

Pass criteria for Phase 6

- ☐ Checkin → Attendance chain works
 - ☐ Roster SPA loads in <5 sec
 - ☐ Roster filter by department works
 - ☐ 0 browser console errors
-

Phase 7 — Regression (~60 min)

Goal: Run all 60+ test cases from 08.2 Test Cases.

Smoke subset (mandatory)

Run T-SMOKE-001 through T-SMOKE-007 (7 cases, ~5 min).

Functional subset (priority-based)

Run these by priority:

Priority	Cases	Count	Time
High	T-EMP-001 to 005, T-SHIFT-001 to 005, T-ATT-001 to 005, T-LEAVE-001 to 005, T-PAY-001 to 005	~25	~30 min
Medium	T-EMP-011 to 015, T-SHIFT-011 to 015, T-ATT-011 to 015, T-LEAVE-011 to 015, T-PAY-011 to 015	~25	~20 min
Low	Remaining	~10	~10 min

Integration subset

Run T-INT-001 to T-INT-004 (4 cases, ~10 min).

Customization subset

Run T-CUST-001 to T-CUST-004 (4 cases, ~5 min).

Pass criteria for Phase 7

- ☐ All High priority pass
- ☐ No more than 2 Medium priority failures
- ☐ Low priority failures noted but not blocking

Triage: P1 = block deploy. P2 = fix before next sprint. P3 = backlog.

Phase 8 — Docs (~15 min)

Goal: Institutional knowledge updated.

LEARNINGS.md updated

- Any new bugs / patterns discovered during testing → add to /root/.openclaw/workspace/.learnings/L
- Format: ****Lesson #NNN:**** <one-line summary> (YYYY-MM-DD). <detail>

MEMORY.md verified

- DB passwords (per env) — re-verify against container env (LEARNINGS #154, #157)
- New env URLs (if any)
- Customization counts (78/189/3/2/2)
- Any new env vars or config

Post-mortem if any incidents

If any test failed → write post-mortem using 05.2 Post-Mortem Template.

Update:

- What happened (timeline)
- Root cause
- Fix
- Lessons learned
- Action items

Pass criteria for Phase 8

- ☐ New lessons added (if any)
 - ☐ MEMORY.md accurate
 - ☐ Post-mortem filed (if incidents)
-

Reporting template

After completion, log results:

Regression Cycle REG-YYYYMMDD-NN

****Date:**** YYYY-MM-DD HH:MM-HH:MM IST
****Tester:**** Venkat (via ERPClaw subagent)
****Env:**** pberpprod (or pberpdev / pberpqa)
****Context:**** <change/PR/quarterly>

Phase Results

Phase	Description	Result	Time	Notes
1	Smoke	☐ / ☐	5 min	
2	Vendor tests	☐ / ☐	30 min	
3	Schema	☐ / ☐	15 min	CF: 78 / PS: 189 / PF: 3 / N: 2 / LH: 2
4	CRUD	☐ / ☐	30 min	
5	Workflow	☐ / ☐	30 min	
6	Integration	☐ / ☐	30 min	
7	Regression	☐ / ☐	60 min	X/Y cases passed
8	Docs	☐ / ☐	15 min	

****Overall:**** ☐ GREEN / ☐ RED
****Blockers:**** <list with severity>
****Sign-off:**** Venkat, YYYY-MM-DD HH:MM IST

Save at: docs/TEST_RESULTS_REG-YYYYMMDD-NN.md or PR comment.

Time estimates

Plan	Phases	Time	Use
Smoke only	1	5 min	Per deploy
Lite	1-6	~2.5 hours	Schema change
Full	1-8	~4 hours	Quarterly, pre-upgrade
Full + Phase 7 (60+ cases)	1-8 + extra Phase 7 detail	1 day	Annual regression
Targeted	Affected phases only	~30 min	Post-incident

Quick start

To run a full 8-phase regression:

1. SSH to VPS

```
ssh -i /root/.openclaw/ssh_key vijay@144.217.163.228
```

2. Phase 1: Smoke (5 min)

```
bash scripts/smoke.sh # if exists, else run curl tests manually
```

3. Phase 2: Vendor tests (30 min)

```
CI=1 bench --site pberpprod.duckdns.org run-tests --app haritha_hospital 2>&1 | tee /tmp/v
```

```
# 4. Phase 3: Schema (15 min)
bench --site pberpprod.duckdns.org console < scripts/check_schema.py

# 5. Phase 4: CRUD (30 min) – manual via API or UI

# 6. Phase 5: Workflow (30 min) – manual

# 7. Phase 6: Integration (30 min) – manual

# 8. Phase 7: Regression (60 min) – execute 08.2 test cases

# 9. Phase 8: Docs (15 min)
# Update LEARNINGS.md, verify MEMORY.md, write report
```

Where to next

- **Test plan strategy?** See 08.1 Test Plan
- **Test case catalog?** See 08.2 Test Cases
- **Post-incident?** See 04.4 Incident Response Plan
- **LEARNINGS.md?** See /root/.openclaw/workspace/.learnings/LEARNINGS.md (especially #48)